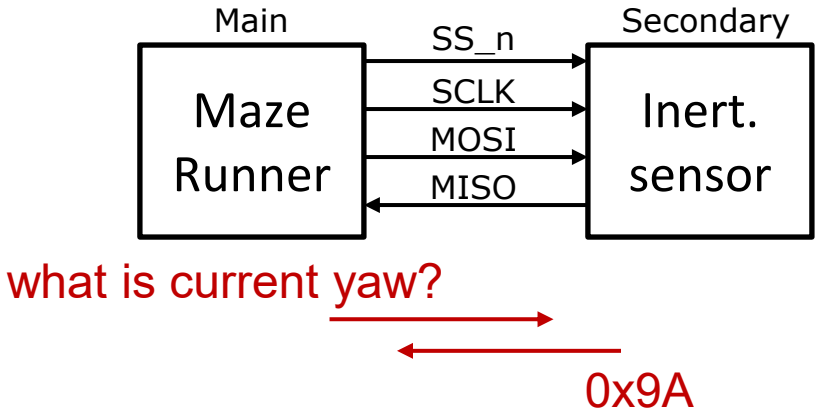
Exercise 15: SPI Transceiver

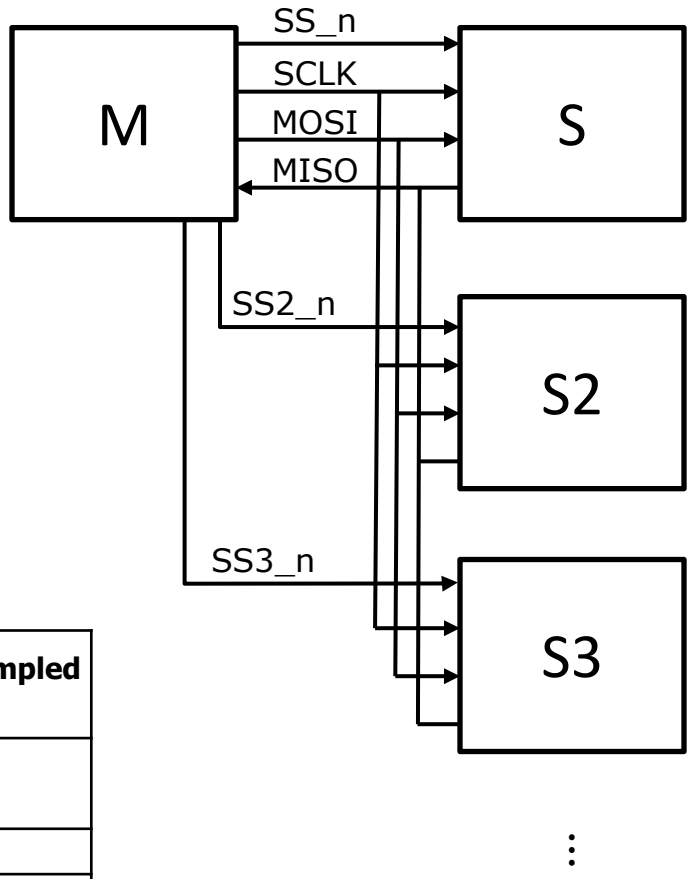
- The MazeRunner needs to get readings from the inertial sensor (MEMs gyro) to figure out its heading
- Communication with the inertial sensor is done using the SPI protocol



- Uses 4-wire, full duplex interface
  - MOSI (Main Out Secondary In) – We drive this to 6-axis inertial sensor
  - MISO (Main In Secondary Out) – Inertial sensor drives this back to us
  - SCLK (Serial clock)
  - SS_n (Active low Secondary Select)

Serial Peripheral Interface (SPI)

- Synchronous serial communication
- 1 Main, typically multiple Secondaries
- 4-wire, full duplex interface
- Many configs / variants:
  - Can have arbitrary data widths
  - Typically MSB first, but not mandated
  - SCLK normally high vs normally low
  - When data shifted & sampled:



SPI mode	Clock polarity (CPOL)	Clock phase (CPHA)	Data shifted out on	Data is sampled on
0	0	0	falling SCLK, and when SS activates	rising SCLK
1	0	1	rising SCLK	falling SCLK
2	1	0	rising SCLK, and when SS activates	falling SCLK
3	1	1	falling SCLK	rising SCLK

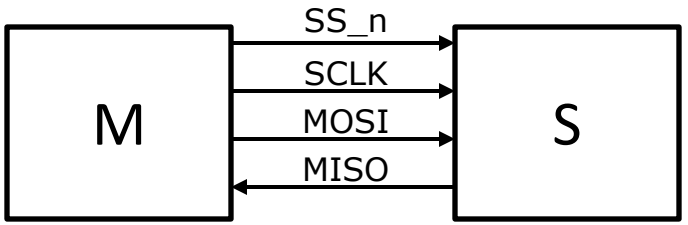
Idle val of SCLK

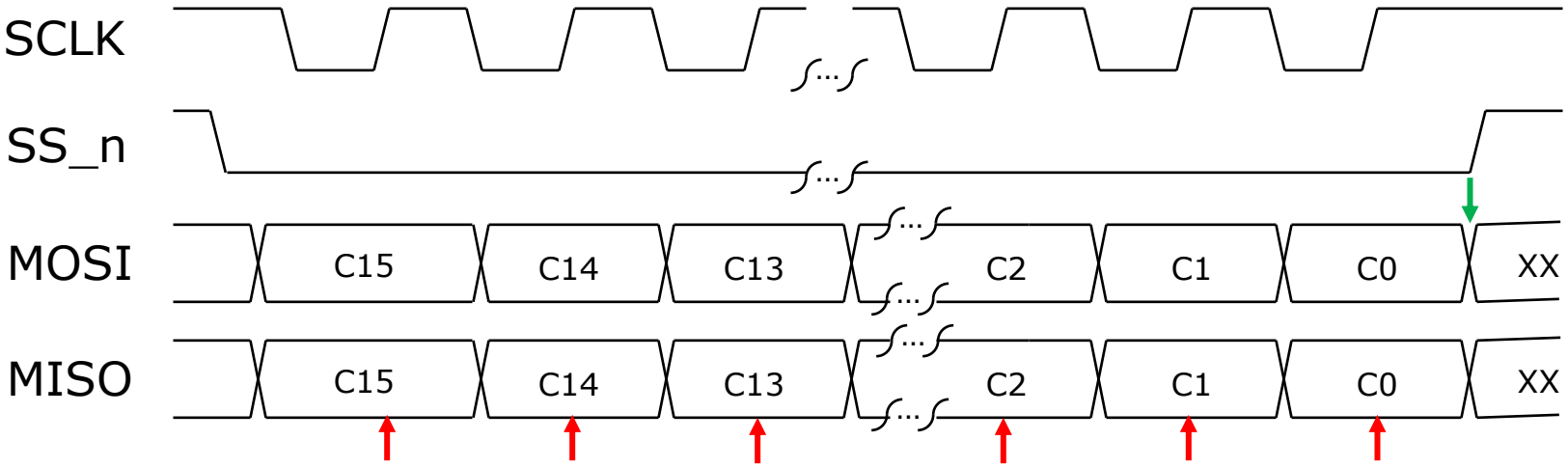
When bits are shifted & sampled

Exercise 15: SPI Transceiver

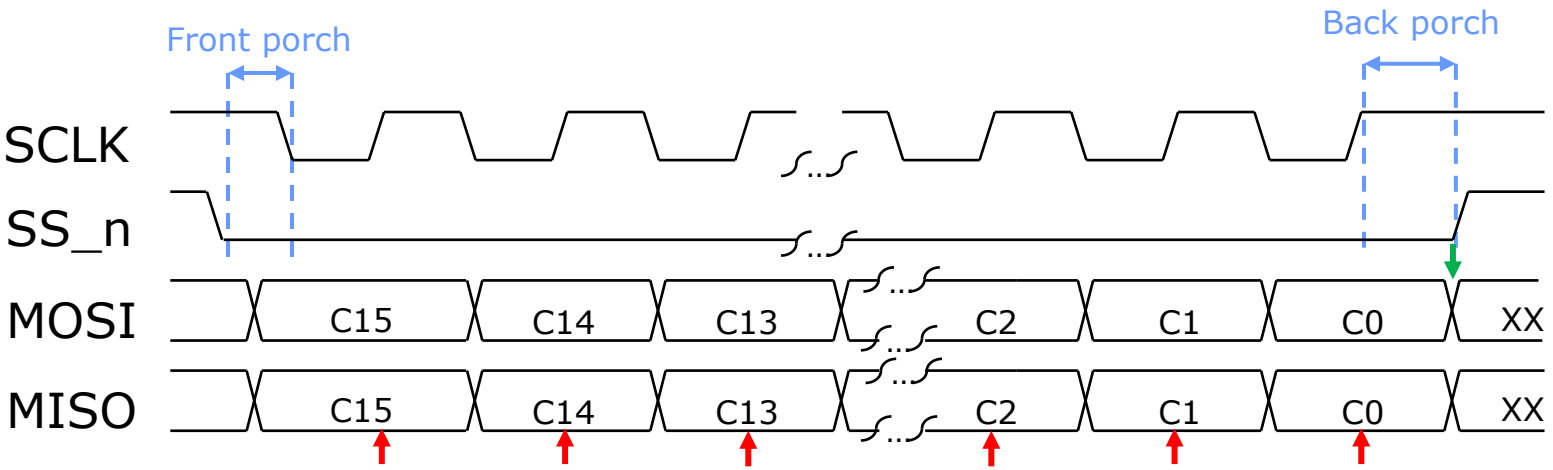
We will stick with the following for the MazeRunner:

- Only 1 secondary on the channel
- 16-bit packets, MSB first
- SCLK normally high, 1/32 of 50MHz system clk
- MOSI shifted on SCLK fall
- MISO sampled on SCLK rise





Detailed Description on Timing



Shown above is a 16-bit SPI packet. The main is changing (shifting) **MOSI** on the falling edge of **SCLK**. The secondary device (6-axis inertial sensor) changes **MISO** on the falling edge too. We sample **MISO** on the rising edge (see red arrows).

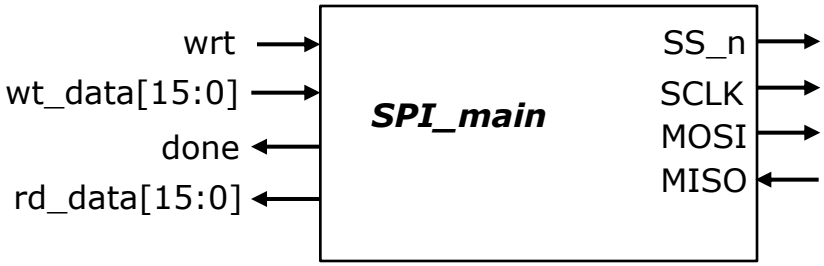
The sampled version of **MISO** in turn gets shifted into our 16-bit shift register (on **SCLK** fall)

When **SS_n** first goes low, there is a short period before **SCLK** goes low. Our 16-bit shift register does not shift on the first fall of **SCLK**. This is called the "front porch".

At the end of the transaction, C0 from the secondary (on **MISO**) is sampled on the last rise of **SCLK**. Then there is a bit of a "back porch" before **SS_n** returns high. When **SS_n** returns high we shift our 16-bit shift register one last time (but prevent the fall of **SCLK**) (see green arrow) so "C0" captured on **SCLK** rise (last red arrow) is shifted into our shift register and we have received 16-bits from the secondary.

Exercise 15: SPI Transceiver

- Implement **SPI_main.sv** with the following interface:



Signal	Dir	Description
clk, rst_n	in	50MHz system clock and reset
SS_n, SCLK, MOSI,MISO	3-out, 1-in	SPI protocol signals outlined above
wrt	in	A high for 1 clock period would initiate a SPI transaction
wt_data[15:0]	in	Data (command) being sent to inertial sensor.
done	out	Asserted when SPI transaction is complete. Should stay asserted till next **wrt**
rd_data[15:0]	out	Data from SPI secondary. For inertial sensor we will only ever use [7:0]

- You will design the datapath and FSM for this module and submit **SPI_main.sv**

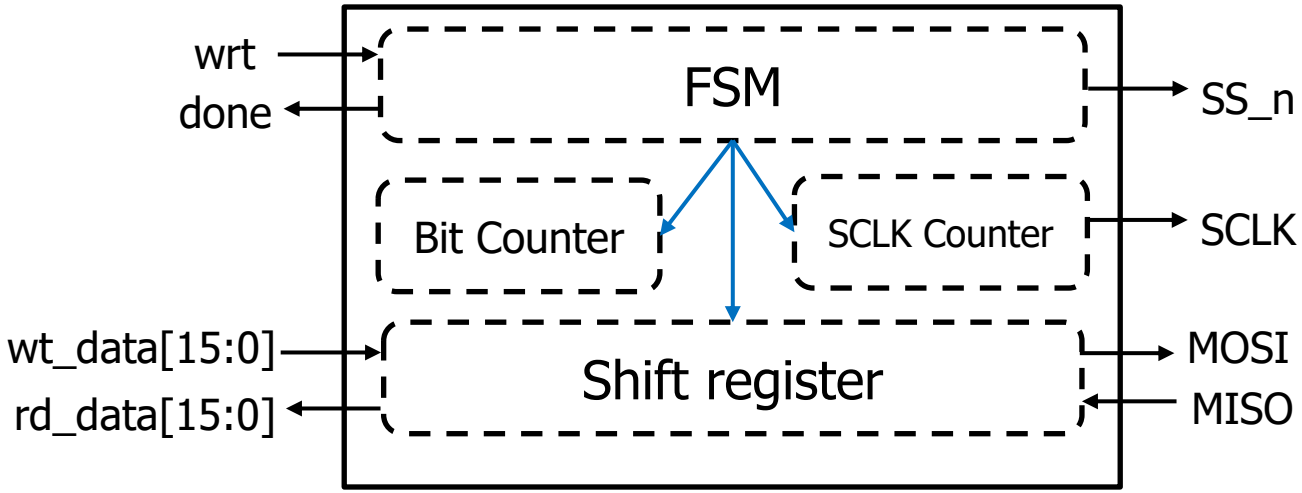
SPI_main Design

What do we need in SPI_main?

- SPI is a serial protocol, so we need to send out / receive parallel data 1 bit at a time
  → need shift register to hold data for send & receive
  → need bit counter to keep track of which bit we are at

- SPI_Main needs to generate SCLK for all Secondaries to observe
  → need counter to act as clock divider

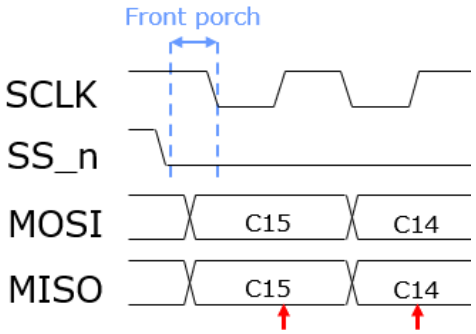
- FSM to manage all the control signals

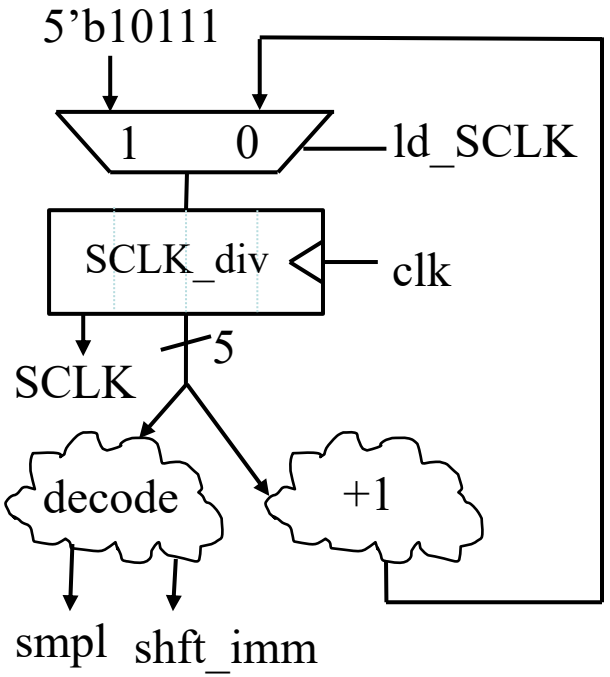


SPI_main Design: SCLK Generation

- SCLK frequency is 1/32 of system clk
  → with a 5-bit clk divider running off clk, SCLK can just be the MSB

- SCLK_div only needs to run during SPI transactions
- Initialize SCLK_div to 5'b10111 (i.e., something close to full) so SCLK will start at 1 and have some delay to create the front porch



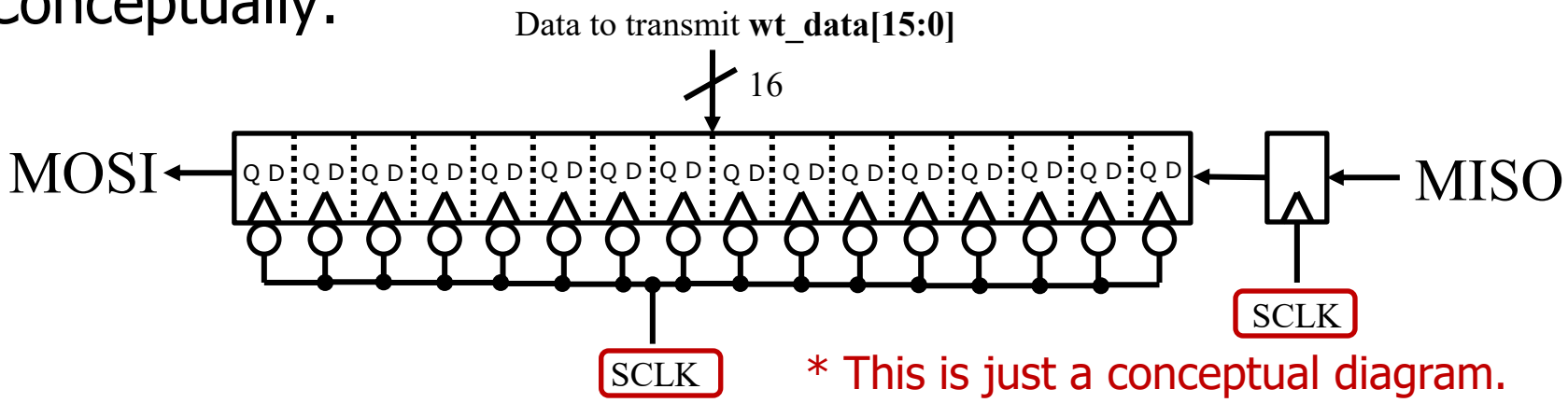


- Decode SCLK_div to generate control signals:
  - smpl – indicate we are sampling MISO next
    (i.e., SCLK rising next clk)
  - shft_imm – indicate shift is imminent
    (i.e., SCLK falling next clk)

SPI_main Design: Shift Register

- SPI is a full duplex interface
  - During operation, in each SCLK cycle, 1 bit is shifted out onto MOSI, 1 bit is shifted in from MISO
    → we can share a 16-bit shift register for both send & receive!

Conceptually:

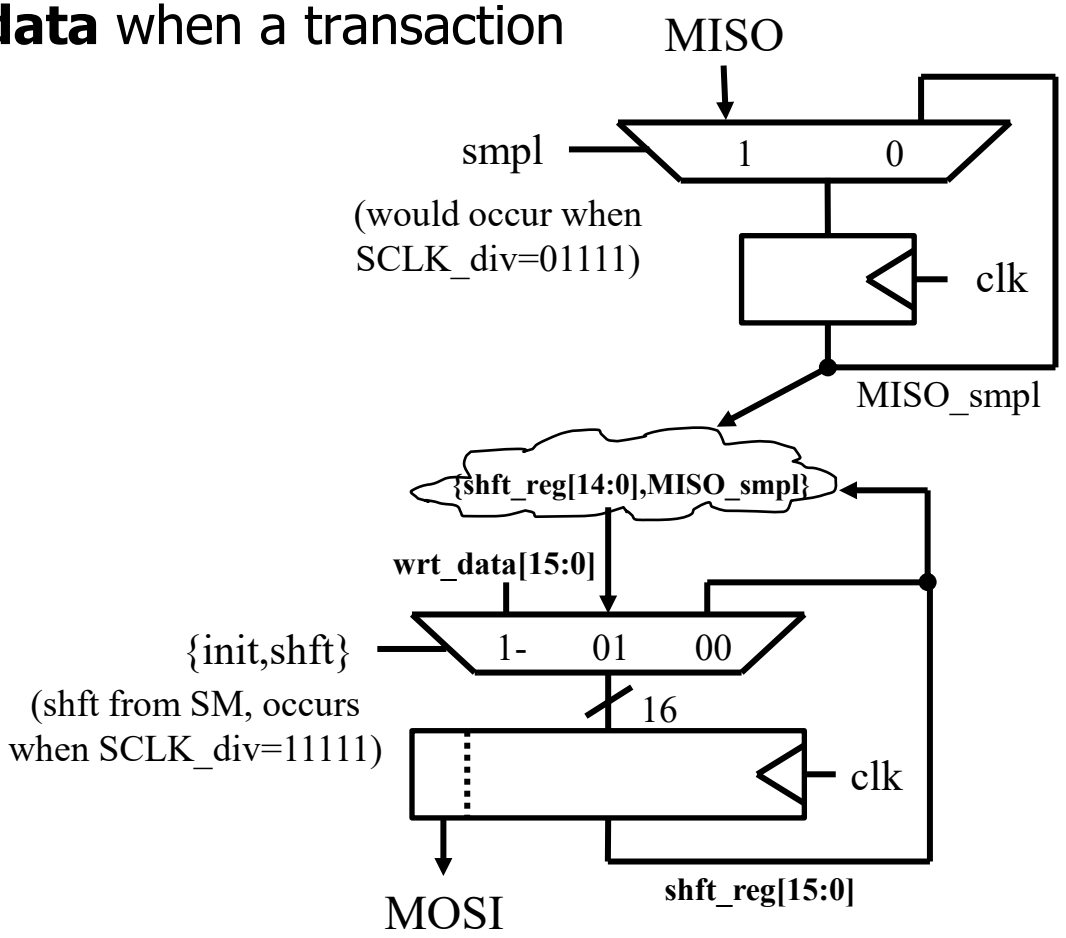


* This is just a conceptual diagram. Make sure your FFs are using clk, not SCLK!

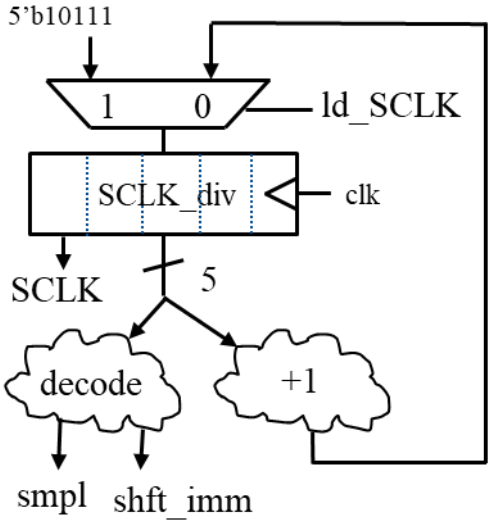
- Parallel load data that we want to transmit
- Shift MSB out & receive data in the LSB at the same time
  - Data on MISO is sampled on the rise of SCLK, then put into shift register on the fall of SCLK

SPI_main Design: Shift Register

- The MSB of shft_reg forms MOSI
- Sample MISO when **smpl** asserted
- Shift sampled bit into LSB of shft_reg when **shft** asserted
- Shft_reg loaded with **wrt_data** when a transaction is initiated
- All FFs are using system clk

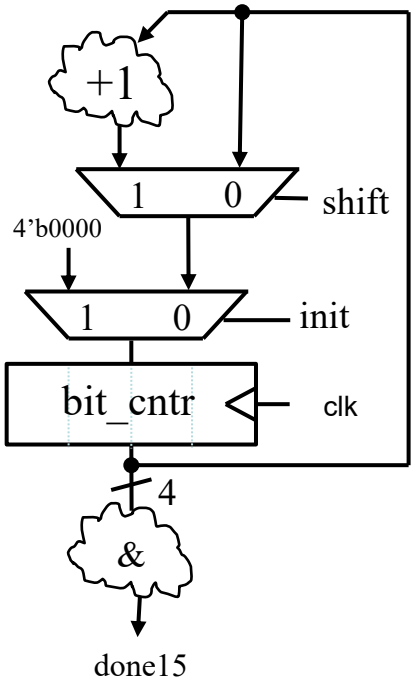


SCLK Generation

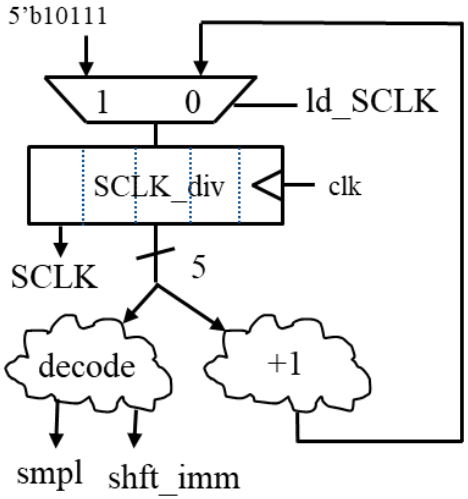


SPI_main Design: Bit Counter & FSM

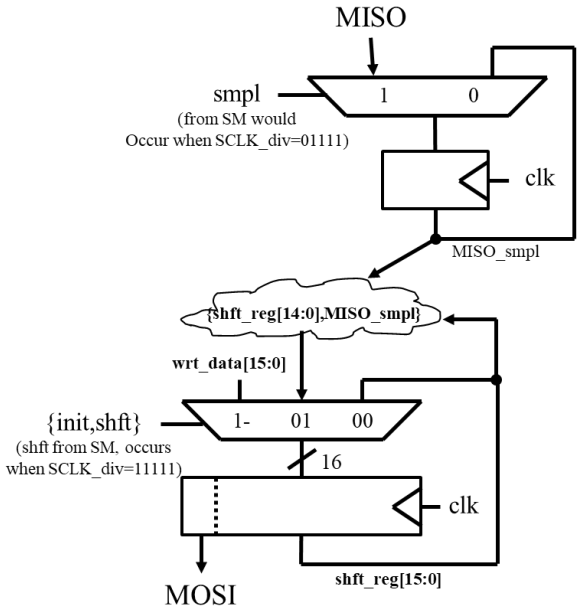
Bit counter



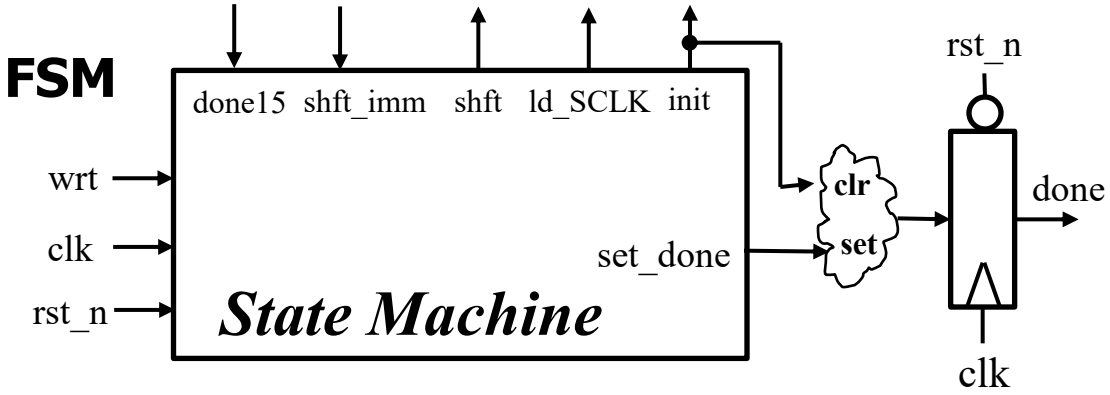
SCLK



Shift register

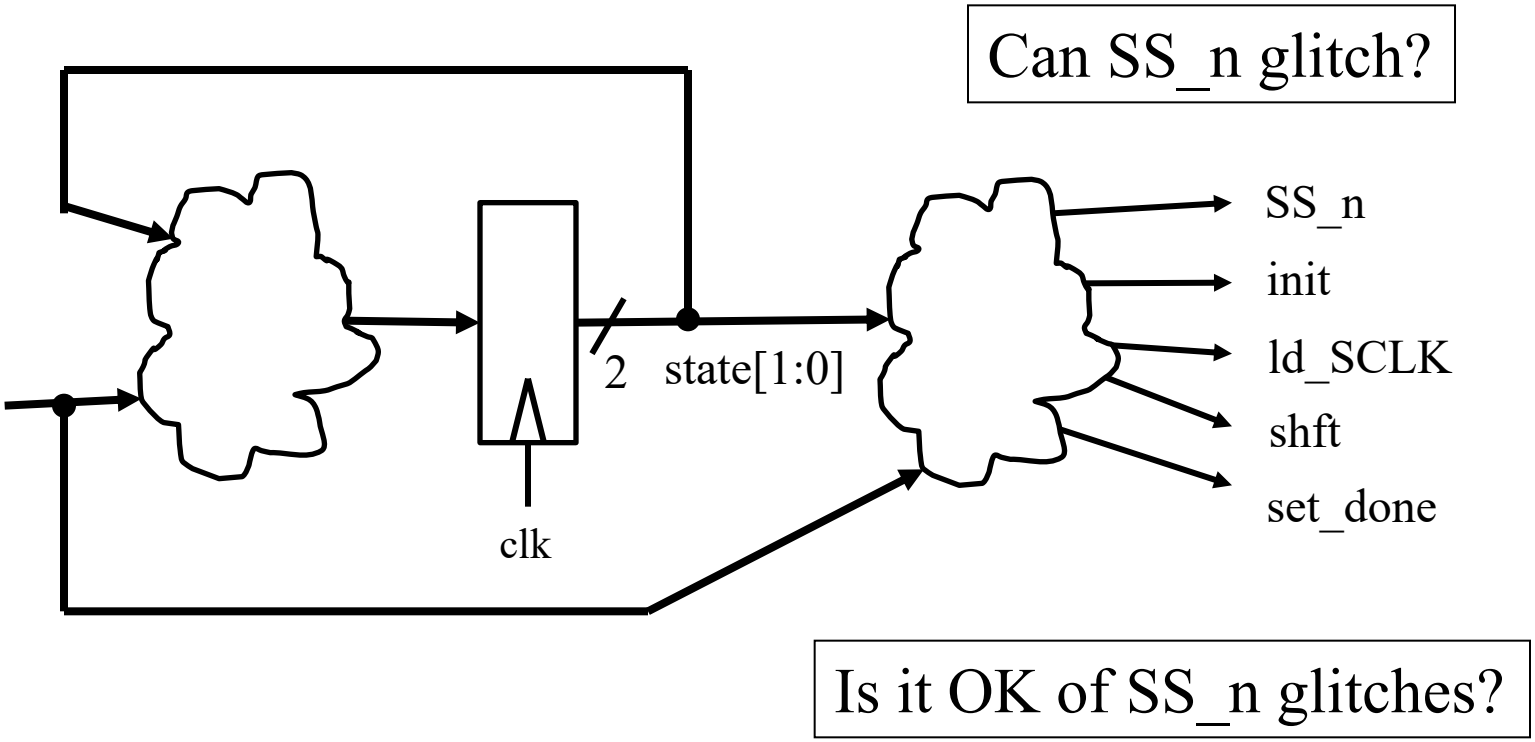


FSM



SPI_main Design: FSM

- FSM is producing a few outputs:



SPI_main Design: FSM

- FSM is producing a few outputs: